

Report - **Project 1**

DETI SHOP

Course: SIO -

Date: Aveiro, 05/09/2024

Student: 106078 : João Vitor B. D. Ferreira

1- Introduction

Deti Shop is a web application where users can buy memorabilia from DETI (Department of Electronics, Telecommunications, and Informatics) at the University of Aveiro. The online shop has a variety of items, including mugs, cups, t-shirts, hoodies

2- System description

This project is a web-based e-commerce platform built using **Flask**. It enables users to register accounts, log in, manage shopping carts, place orders, manage user profiles, and upload profile images. The application employs **SQLite** as its database, and it follows the **blueprint architecture** and **application factory pattern** of Flask to organize the various modules of the system.

The system was structured to be modular, making it easier to maintain and scale. Several common web security vulnerabilities (CWEs) have been intentionally implemented or may exist due to a lack of proper security practices.

3- Security Weaknesses (CWEs)

1. CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')
2. CWE-89: Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')
3. CWE-620: Unverified Password Change
4. CWE-916: Use of Password Hash with Insufficient Computational Effort
5. CWE-287: Improper Authentication
6. CWE-521: Weak Password Requirements
7. CWE-20: Improper Input Validation
8. CWE-434: Unrestricted Upload of File with Dangerous Type
9. CWE-22: Path Traversal
10. CWE-285: Improper Authorization**(Subtle Vulnerability)**
11. CWE-352: Cross-Site Request Forgery (CSRF)

4- Vulnerabilities

4.1- CWE-89: Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')

Exploitation 1

Description:

The SQL Injection vulnerability can be exploited in the **search route** of the application, where the SQL query is constructed directly with user input without proper protection. This allows an attacker to inject malicious SQL code into the query, compromising the security of the database.

Impact:

If exploited, this vulnerability allows an attacker to:

Execute arbitrary SQL commands, such as modifying, deleting, or inserting data.

Completely compromise the integrity and availability of the database.

Code:

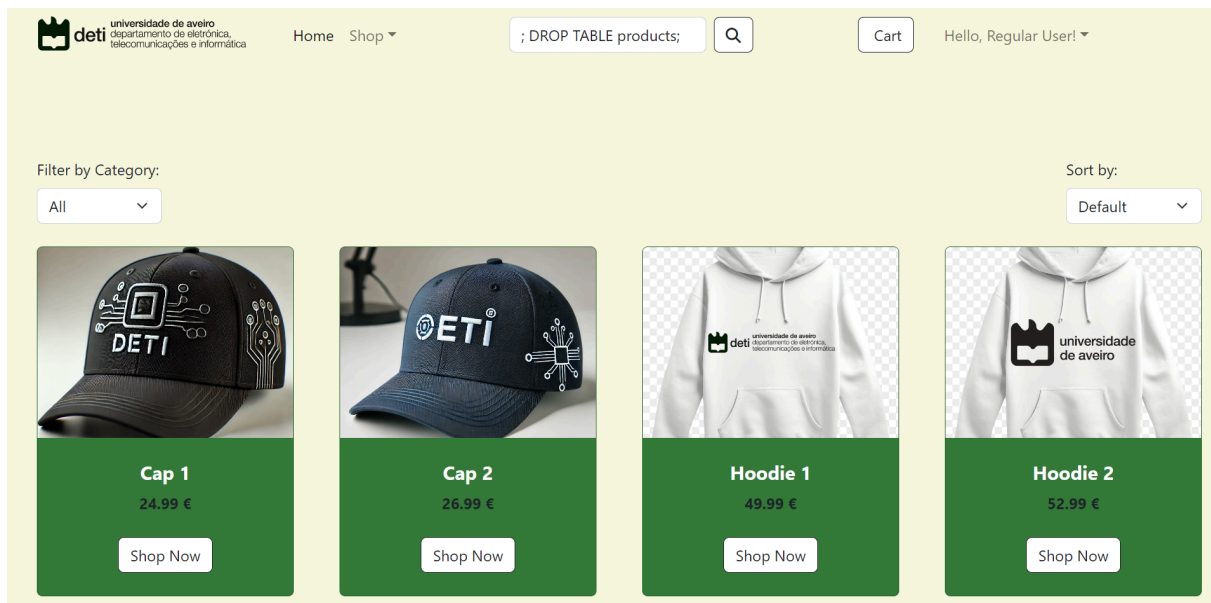
```
@products.route('/search')
def search():
    try:
        user_id = session['user_id']
        user = get_user_by_id(user_id)
    except:
        user = None
    query = request.args.get('query')
    if query:
        sql_query = text(f"SELECT * FROM products WHERE name LIKE '%{query}%'")
        products = db.session.execute(sql_query).fetchall()
    else:
        products = []
    return render_template('search_results.html', products=products, query=query, user=user)
```

Exploitation of the Vulnerability:

Deleting the Products Table:

An attacker can inject malicious SQL commands to delete the products table from the database, using the following input on the search bar: **""; DROP TABLE products; --"**

As a result, the products table will be deleted, which may cause a total loss of product data in the database or an internal error.



Vulnerability Fix:

To fix this vulnerability, parameterized queries were used in the safe version of the application to safely handle user inputs and prevent SQL injection attacks. The user input is safely inserted into the query by escaping any dangerous characters that could be used to manipulate the SQL query, which prevents malicious input from being interpreted as part of the SQL command.

```
@products.route('/search')
def search():
    try:
        user_id = session['user_id']
        user = get_user_by_id(user_id)
    except:
        user = None

    query = request.args.get('query')
    if query:
        products = Product.query.filter(Product.name.ilike(f"%{query}%")).all()
    else:
        products = []
    return render_template('search_results.html', products=products, query=query, user=user)
```

Exploitation 2

Description:

The SQL Injection vulnerability can be exploited in the **user registration route**, where the SQL query for checking if an email or username already exists is constructed directly using user input without proper protection. This allows an attacker to inject malicious SQL code into the query, bypassing the verification process and registering a new account, even if the email or username already exists in the database.

Impact:

If exploited, this vulnerability allows an attacker to:

Bypass the registration process, enabling the creation of new accounts using an already registered email or username.

Potentially interfere with user data and manipulate the registration logic, leading to account duplication or data corruption.

Can lead to an internal server error.

Code:

```
# SQL Injection vulnerability in the user check query
query = text(f"SELECT * FROM users WHERE email = '{email}' OR username = '{username}'")
existing_user = db.session.execute(query).fetchone()

if existing_user:
    flash('Email or username already exists. Please choose another.', 'error')
    return redirect(url_for('auth.register'))
```

Exploitation of the Vulnerability:

Bypassing Registration Verification:

It's possible to inject malicious SQL code to bypass the email and username verification when registering a new account by using the input **" OR '1'='1"** in the email field of the registration form.

Vulnerability Fix

By utilizing SQLAlchemy's filter() method in the safe version, the application prevents any malicious input from being interpreted as part of the SQL command. The SQLAlchemy ORM ensures that all input values, such as email and username, are safely passed as parameters, protecting the database from SQL injection attacks.

```
existing_user = User.query.filter((User.email == email) | (User.username == username)).first()
if existing_user:
    flash('Email or username already exists. Please choose another.', 'error')
    return redirect(url_for('auth.register'))
```

4.2- CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')

Exploitation: XSS Vulnerability in Product Reviews

Description:

The Cross-Site Scripting (XSS) vulnerability exists in the **product reviews**, where a user can submit comments and these comments are rendered without proper sanitization or escaping. An attacker can exploit this by injecting malicious JavaScript code through the review comment field, which will be executed in the browser of any user who views the product page.

Although Jinja2 sanitizes inputs automatically, and it's fairly used in this application, for educational purposes, this was disabled for being possible to exploit this CWE.

Impact:

If exploited, this vulnerability allows an attacker to:

Execute arbitrary JavaScript in the context of the victim's browser.

Steal sensitive user information, such as session tokens or credentials.

Deface the application by injecting unwanted content or redirecting users to malicious sites.

Launch further attacks by infecting other users who view the malicious content.

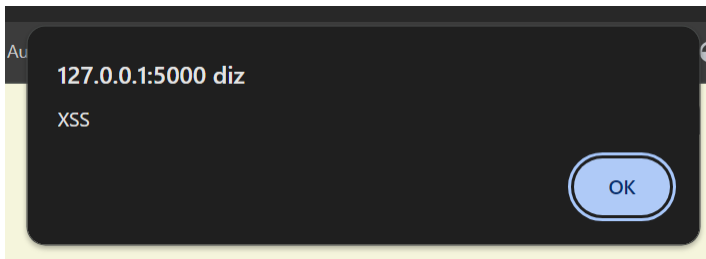
Code:

```
<div class="list-group-item list-group-item-action mb-3 rounded shadow-sm review-item" style="background-color: #ffffff;">
  <div class="d-flex w-100 justify-content-between">
    <h5 class="mb-1">Rating: {{ review.rating }} / 5</h5>
    <small>{{ review.date.strftime('%d-%m-%Y') }}</small>
  </div>
  <p class="mb-1">{{ review.comment | safe }}</p>
  <small>Reviewed by: {{ review.user.username }}</small>
</div>
```

Exploitation of the Vulnerability:***Injecting Malicious Script:***

An attacker can submit a comment such as:

```
<script>alert('XSS');</script>
```

**Vulnerability Fix:**

To fix this vulnerability, the safe version escapes the user input before rendering it on the page. App_sec contains:

```
<p class="mb-1">{{ review.comment | e }}</p>
```

4.3- CWE-620: Unverified Password Change

Exploitation : Unverified password change

Description:

The password change functionality in the application allows users to change password without requiring the current password. This vulnerability could be exploited by an attacker with access to a user's session, allowing them to change the account's password without the user's knowledge or consent.

Impact:

If exploited, this vulnerability allows an attacker to:

Change the user's password without needing to know the current password.

Compromise the user account entirely, locking the original user out of their account.

Gain unauthorized access to sensitive areas of the application by modifying the login credentials.

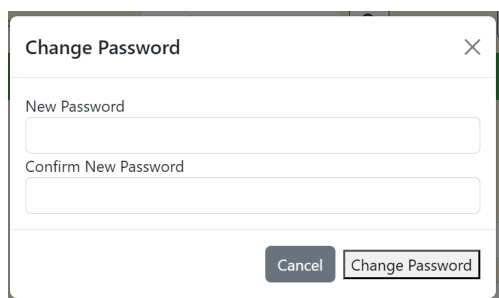
Code:

```
<div class="modal-body">
  <div class="form-group">
    <label for="new_password">New Password</label>
    <input type="password" class="form-control" name="new_password" id="new_password" required>
  </div>
  <div class="form-group">
    <label for="confirm_password">Confirm New Password</label>
    <input type="password" class="form-control" name="confirm_password" id="confirm_password" required>
  </div>
</div>
<div class="modal-footer">
  <button type="button" class="btn btn-secondary" data-bs-dismiss="modal">Cancel</button>
  <button type="submit" the="btn btn-primary">Change Password</button>
</div>
```

Exploitation of the Vulnerability:

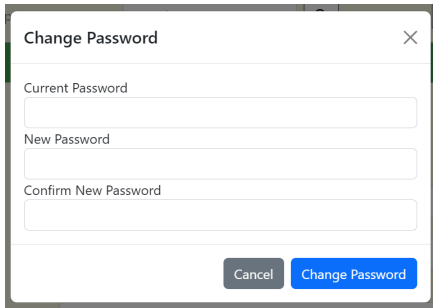
Changing password:

An attacker could gain access to a user's session (through session hijacking or other means) and change the user's password without needing to know the current password.



Vulnerability Fix:

To fix this vulnerability, the safe version requires the user to input their current password before allowing a password change. This ensures that even if an attacker has access to a session, they will not be able to change the password without knowing the current one.



The image shows a 'Change Password' dialog box. It has a title bar with a close button (X). Inside, there are three text input fields labeled 'Current Password', 'New Password', and 'Confirm New Password'. At the bottom right, there are two buttons: a grey 'Cancel' button and a blue 'Change Password' button.

```
@profile.route('/profile/change_password', methods=['POST'])
@login_required
def change_password():
    user_id = session.get('user_id')
    user = get_user_by_id(user_id)

    current_password = request.form.get('current_password')
    new_password = request.form.get('new_password')
    confirm_password = request.form.get('confirm_password')

    if not user.check_password(current_password):
        flash('Current password is incorrect.', 'error')
        return redirect(url_for('profile.profile_page'))

    if new_password != confirm_password:
        flash('New password and confirmation do not match.', 'error')
        return redirect(url_for('profile.profile_page'))

    user.set_password(new_password)
    db.session.commit()

    flash('Password changed successfully!', 'success')
    return redirect(url_for('profile.profile_page'))
```

4.4- CWE-916: Use of Password Hash With Insufficient Computational Effort

Exploitation: Use of Insecure Hashing Algorithm (MD5)

Description:

The application uses the **MD5 hashing algorithm** to hash user passwords. MD5 is a weak hashing algorithm that is susceptible to **collision attacks**, where two different inputs can produce the same hash output. MD5 also lacks computational complexity, making it vulnerable to **brute force**. Impact:

If exploited, this vulnerability allows an attacker to:

Compromise the integrity of the authentication system, potentially gaining unauthorized access to user accounts.

Obtain hashed passwords and reverse them back to plaintext.

Code:

```
def set_password(self, password):
    self.password = hashlib.md5(password.encode('utf-8')).hexdigest()

def check_password(self, password):
    return self.password == hashlib.md5(password.encode('utf-8')).hexdigest()
```

Exploitation of the Vulnerability:

An attacker can exploit this vulnerability by:

Brute Force Attack: Using rainbow tables or precomputed MD5 hashes, an attacker could reverse the hash back to the original password.

Collision Attack: Two different passwords could generate the same MD5 hash, allowing an attacker to use one password to authenticate as another user.

Vulnerability Fix:

To mitigate this vulnerability, the use of **MD5** was replaced by **PBKDF2**, a password hashing algorithm specifically designed to be resistant to brute-force attacks through the use of **key stretching**. PBKDF2 allows us to hash passwords with multiple iterations and a **unique salt** to make it computationally expensive to crack the hashes.

```

from werkzeug.security import generate_password_hash, check_password_hash

def set_password(self, password):
    self.password = generate_password_hash(password, method='pbkdf2:sha256', salt_length=16)

def check_password(self, password):
    return check_password_hash(self.password, password)

```

4.5- CWE-287: Improper Authentication

Exploitation: Admin Access Without Proper Authorization

Description:

The system does not verify user permissions when accessing the admin area. Any authenticated user can access the admin page simply by navigating to /admin, regardless of whether they have administrative privileges.

If exploited, this vulnerability allows an attacker to:

Access restricted administrative functionality, such as user management, product management, or sensitive configurations.

Modify, delete, or view sensitive information that should only be available to administrators.

Compromise the security and integrity of the system by performing actions typically reserved for administrators.

Code:

```

@admin_bp.route('/admin', methods=['GET'])
@login_required
def admin():
    return redirect(url_for('admin.inventory_management'))

```

```

@admin_bp.route('/admin/inventory', methods=['GET'])
@login_required
def inventory_management():

```

```

@admin_bp.route('/admin/inventory/update/<int:product_id>', methods=['POST'])
@login_required
def update_inventory(product_id):

```

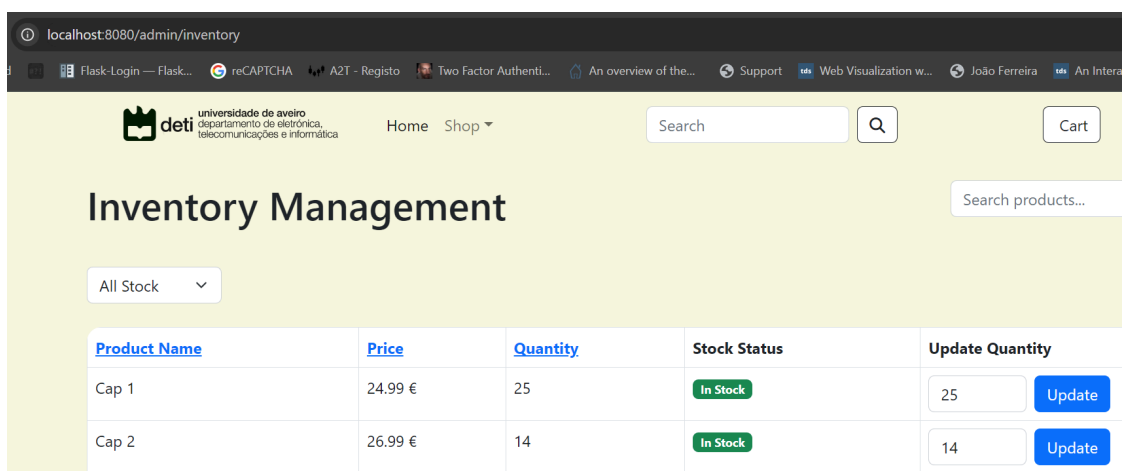
Exploitation of the Vulnerability:

An attacker can exploit this vulnerability by:

Logging in as a regular user.

Manually navigating to the /admin URL.

The application does not verify whether the user has administrative privileges and allows access to the admin area.

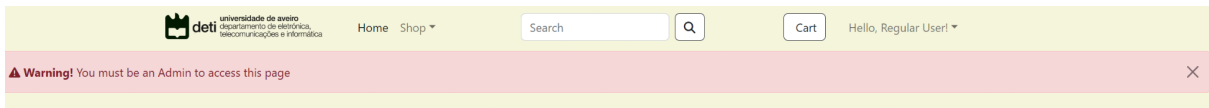


Vulnerability Fix:

To fix this vulnerability, a (RBAC) system was implemented. Only users with administrative privileges are allowed to access all the /admin route.

```
@admin_bp.route('/admin/inventory', methods=['GET'])
@login_required
@admin_required
def inventory_management():
```

```
def admin_required(func):
    @wraps(func)
    def decorated_function(*args, **kwargs):
        if not User.query.filter_by(id= session['user_id']).first().is_admin() :
            flash('You must be an Admin to access this page', 'error')
            return redirect(url_for('auth.login'))
        return func(*args, **kwargs)
    return decorated_function
```



4.6- CWE-521: Weak Password Requirements

Exploitation : Weak Password Policy Allowing Insecure Passwords

Description:

Without minimum password complexity rules, users can choose weak passwords that are easy to guess or crack, exposing the system to brute force and dictionary attacks.

Impact:

If exploited, this vulnerability allows an attacker to:

Easily guess or crack weak passwords using brute force or dictionary attacks.

Gain unauthorized access to user accounts and sensitive information.

Compromise user accounts and escalate privileges.

Code:

```
@auth.route('/register', methods=['GET', 'POST'])
def register():
    if request.method == 'POST':
        name = request.form['name']
        email = request.form['email']
        username = request.form['username']
        password = request.form['password']
        confirm_password = request.form['confirm_password']
        phone = request.form['phone']
        file = request.files['profile_picture']

        if password != confirm_password:
            flash('Passwords do not match. Please try again.', 'error')
            return redirect(url_for('register'))
```

Exploitation of the Vulnerability:

An attacker can exploit this vulnerability by testing commonly used weak passwords, such as "123456" or "adminpass". Since the application does not enforce strong password requirements, users can select these weak passwords, making it trivial for attackers to compromise accounts.

Vulnerability Fix:

In the safe version, a method for password validation was created, it requires more complexity from

the user input. This ensures that the password meets the minimum security requirements for length, special characters, upper/lowercase letters, and numbers. This method was also used in the **profile page form**, where the user can update his password.

```
def validate_password(password, confirm_password):
    if password != confirm_password:
        flash('Passwords do not match. Please try again.', 'error')
        return False

    if len(password) < 8:
        flash("Password must be at least 8 characters long.", "error")
        return False

    if not re.search(r"[A-Z]", password):
        flash("Password must contain at least one uppercase letter.", "error")
        return False

    if not re.search(r"[a-z]", password):
        flash("Password must contain at least one lowercase letter.", "error")
        return False

    if not re.search(r"\d", password):
        flash("Password must contain at least one number.", "error")
        return False

    if not re.search(r"[!@#$%^&*(),.?\":{}|<>]", password):
        flash("Password must contain at least one special character.", "error")
        return False

    return True
```

```
@auth.route('/register', methods=['GET', 'POST'])
def register():
    if request.method == 'POST':
        name = request.form['name']
        email = request.form['email']
        username = request.form['username']
        password = request.form['password']
        confirm_password = request.form['confirm_password']
        phone = request.form['phone']
        file = request.files['profile_picture']

        if password != confirm_password:
            flash('Passwords do not match. Please try again.', 'error')
            return redirect(url_for('register'))

        if not validate_password:
            return redirect(url_for('auth.login'))
```

```
@profile.route('/profile/change_password', methods=['POST'])
@login_required
def change_password():
    user_id = session.get('user_id')
    user = get_user_by_id(user_id)

    current_password = request.form.get('current_password')
    new_password = request.form.get('new_password')
    confirm_password = request.form.get('confirm_password')

    if not user.check_password(current_password):
        flash('Current password is incorrect.', 'error')
        return redirect(url_for('profile.profile_page'))

    if not validate_password(new_password, confirm_password):
        return redirect(url_for('profile.profile_page'))

    user.set_password(new_password)
    db.session.commit()

    flash('Password changed successfully!', 'success')
    return redirect(url_for('profile.profile_page'))
```

4.7- CWE-20: Improper Input Validation

Exploitation : Lack of Input Validation in User Registration

Description:

The absence of proper validation mechanisms for fields like email, phone number, and profile picture allows users to submit invalid or potentially harmful data, leading to security risks and functional issues.

Impact:

If exploited, this vulnerability allows an attacker to:

Submit invalid or harmful data into the system, such as SQL injections or XSS attacks.

Manipulate user inputs to bypass certain checks or submit malformed data, potentially leading to security breaches.

Exploit weak input validation to gain unauthorized access to certain functionalities or inject harmful code into the system.

Code:

```
@auth.route('/register', methods=['GET', 'POST'])
def register():
    if request.method == 'POST':
        name = request.form['name']
        email = request.form['email']
        username = request.form['username']
        password = request.form['password']
        confirm_password = request.form['confirm_password']
        phone = request.form['phone']
        file = request.files['profile_picture']

        if password != confirm_password:
            flash('Passwords do not match. Please try again.', 'error')
            return redirect(url_for('register'))
```

Exploitation of the Vulnerability:

An attacker could exploit this vulnerability by submitting malicious inputs into the registration form. Since there is no proper validation of fields such as email, phone number, or file type, the attacker could submit data that the application is not prepared to handle.

Vulnerability Fix:

To fix this vulnerability, input validation was implemented on all fields to ensure that data

submitted by users is valid, safe, and expected. Each field has clear rules about what is allowed and should be sanitized before being processed.

```
def validate_email(email):
    email_regex = r'^[a-zA-Z0-9_+-.]+@[a-zA-Z0-9-]+\.[a-zA-Z0-9-]+\.$'
    if not re.match(email_regex, email):
        flash('Invalid email format.', 'error')
        return False
    return True

def validate_phone(phone):
    phone_regex = r'^\+?[0-9]{7,15}$'
    if not re.match(phone_regex, phone):
        flash('Invalid phone number format.', 'error')
        return False
    return True

ALLOWED_EXTENSIONS = {'png', 'jpg', 'jpeg', 'gif'}

def allowed_file(filename):
    return '.' in filename and filename.rsplit('.', 1)[1].lower() in ALLOWED_EXTENSIONS
```

4.8- CWE-89:CWE-434: Unrestricted File Upload

Exploitation 1

Description:

The application allows users to upload profile pictures during registration without validating the file type or checking for potentially harmful content. This creates an opportunity for attackers to upload arbitrary files, including those that could be used to execute code or manipulate the system.

Impact:

If exploited, this vulnerability allows an attacker to:

Upload harmful files, such as scripts, that could be used to execute commands or gain unauthorized access to the system.

Store malicious files on the server, potentially compromising the system or other users.

Code:

```
file = request.files['profile_picture']
if file and file.filename != '':
    filename = file.filename
    file_path = os.path.join(Config.UPLOAD_FOLDER, filename)
    file.save(file_path)
else:
    file_path = os.path.join(Config.UPLOAD_FOLDER, 'default_user.jpg')
```

Exploitation of the Vulnerability:

An attacker could upload any file, including a malicious script or executable, without any checks. This could allow attackers to upload harmful files that could be used for various exploits such as privilege escalation or denial of service.

Vulnerability Fix:

To mitigate this vulnerability, a method for file validation was created. It restricts the types of files that can be uploaded and ensures that only allowed file types (e.g., images) are accepted.

```
ALLOWED_EXTENSIONS = {'png', 'jpg', 'jpeg', 'gif'}

def allowed_file(filename):
    return '.' in filename and filename.rsplit('.', 1)[1].lower() in ALLOWED_EXTENSIONS

if file and allowed_file(file.filename):
    filename = secure_filename(file.filename)
    file_path = os.path.join(Config.UPLOAD_FOLDER, filename)
    file.save(file_path)
else:
    flash('Invalid file type. Please upload an image.', 'error')
    return redirect(url_for('auth.register'))
```

By validating the file type and only allowing certain extensions (e.g., .png, .jpg, .jpeg, .gif), the application restricts uploads to safe file types and prevents the upload of potentially harmful files.

4.9- CWE-22: Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal')

Exploitation : File Path Manipulation

Description:

The application does not sanitize the filename before saving the uploaded file. This creates a Path Traversal vulnerability, allowing attackers to manipulate the file path and upload files outside the intended directory.

Impact:

If exploited, this vulnerability allows an attacker to:

Save files in arbitrary locations on the server, potentially overwriting critical system files or

placing files in sensitive locations.

Gain unauthorized access to other users' files or system files by manipulating the file path.

Code:

```
file = request.files['profile_picture']
if file and file.filename != '':
    filename = file.filename
    file_path = os.path.join(Config.UPLOAD_FOLDER, filename)
    file.save(file_path)
else:
    file_path = os.path.join(Config.UPLOAD_FOLDER, 'default_user.jpg')
```

Exploitation of the Vulnerability:

Deleting the Products Table:

An attacker could submit a filename like `../../../../etc/passwd` to traverse the directory structure and upload files outside the intended folder, potentially overwriting important system files.

Vulnerability Fix:

To fix this vulnerability, the safe version sanitizes the filename using `secure_filename` from Werkzeug package to ensure that only valid filenames are accepted.

```
if file and allowed_file(file.filename):
    filename = secure_filename(file.filename)
    file_path = os.path.join(Config.UPLOAD_FOLDER, filename)
    file.save(file_path)
else:
    flash('Invalid file type. Please upload an image.', 'error')
    return redirect(url_for('auth.register'))
```

4.10 -CWE-285: Improper Authorization**(Subtle Vulnerability)**

Exploitation : Unauthorized Access to Orders via URL Manipulation

Description:

Users can access and view order details or repeat orders by manipulating the `order_id` in the URL. Although the routes are protected by `@login_required`, the system does not verify if the user is the rightful owner of the order. This lack of authorization checks allows attackers to access or repeat orders that belong to other users.

Impact:

If exploited, this vulnerability allows an attacker to:

View other users' orders: Any user can access another user's order details by guessing or manipulating the **order_id**.

Repeat orders of other users: A malicious user could add items from another user's order to their own cart, potentially affecting the integrity of the system.

Code:

```
@orders.route('/order/confirmation/<int:order_id>')
@login_required
def order_confirmation(order_id):
    user_id = session.get('user_id')
    user = get_user_by_id(user_id)
    order = Order.query.get_or_404(order_id)
    order_items = order.items

    subtotal, tax, shipping, total = calculate_totals(order_items)

    return render_template('order_confirmation.html',
                           order=order,
                           order_items=order_items,
                           subtotal=subtotal,
                           tax=tax,
                           shipping=shipping,
                           total=total,
                           user=user)
```

```
@orders.route('/order/repeat/<int:order_id>')
@login_required
def repeat_order(order_id):
    user_id = session.get('user_id')
    order = Order.query.get_or_404(order_id)
    order_items = order.items

    cart = Cart.query.filter_by(user_id=user_id).first()

    for item in order_items:
        existing_cart_item = CartItem.query.filter_by(cart_id=cart.id, product_id=item.product_id).first()
```

Exploitation of the Vulnerability:

An attacker can exploit this vulnerability by modifying the **order_id** in the URL, then have access to an order details made by another user without proper authorization.

For example: **"/order/confirmation/1"**

Order Confirmation

Order Summary

Order Number: 1

Order Date: 05 September 2024, 06:52

Shipping Address:

123 Street, City

Items in Your Order



Hoodie 1

Comfortable hoodie for everyday wear....

\$49.99

Quantity: 1



Mug 1

High-quality ceramic mug....

\$19.98

Quantity: 2

Similarly, by manipulating the `order_id` in the repeat order URL, the attacker can repeat someone else's order:

For example: **“/order/repeat/1”**



universidade de aveiro
 departamento de eletrónica,
 telecomunicações e informática

[Home](#)
[Shop](#)

Hello, João Vitor Ferreira! ▾

Your Shopping Cart



Hoodie 1
 Comfortable hoodie for everyday wear....
Price: \$49.99
 Quantity:

Remove



Mug 1
 High-quality ceramic mug....
Price: \$9.99
 Quantity:

Remove

Order Summary

Subtotal: 69.97 €

Tax: 7.00 €

Shipping: 10.00 €

Total: 86.97 €

Proceed to Checkout

Vulnerability Fix:

To fix this vulnerability, the query was modified to ensure that the **order_id** being accessed belongs to the authenticated user. This can be done by verifying that **order.user_id** matches the **session['user_id']** before returning the order details.

```
order = Order.query.filter_by(id=order_id, user_id=user_id).first_or_404()
```

```
@orders.route('/order/confirmation/<int:order_id>')
@login_required
def order_confirmation(order_id):
    user_id = session.get('user_id')
    user = get_user_by_id(user_id)

    order = Order.query.filter_by(id=order_id, user_id=user_id).first_or_404()
    order_items = order.items

    subtotal, tax, shipping, total = calculate_totals(order_items)

    return render_template('order_confirmation.html',
                           order=order,
                           order_items=order_items,
                           subtotal=subtotal,
                           tax=tax,
                           shipping=shipping,
                           total=total,
                           user=user)
```

```
@orders.route('/order/repeat/<int:order_id>')
@login_required
def repeat_order(order_id):
    user_id = session.get('user_id')
    order = Order.query.filter_by(id=order_id, user_id=user_id).first_or_404()
    order_items = order.items

    cart = Cart.query.filter_by(user_id=user_id).first_or_404()

    for item in order_items:
        existing_cart_item = CartItem.query.filter_by(cart_id=cart.id, product_id=item.product_id).first()

        if existing_cart_item:
            existing_cart_item.quantity += item.quantity
        else:
            new_cart_item = CartItem(
                cart_id=cart.id,
                product_id=item.product_id,
                quantity=item.quantity
            )
            db.session.add(new_cart_item)

    db.session.commit()

    return redirect(url_for('cart.cart_page'))
```

4.11- CWE-352: Cross-Site Request Forgery (CSRF)

Exploitation 1

Description:

All forms in this application are vulnerable to **CSRF** attacks. CSRF occurs when a malicious website, email, or program causes a user's browser to perform an unwanted action on a different site where the user is **authenticated**. The vulnerability arises because the application does not validate that the request is coming from the form on the original website.

Impact:

If exploited, this vulnerability allows an attacker to:

Perform unauthorized actions on behalf of an authenticated user without their knowledge

or consent.

Code:

```
<form action="{{ url_for('cart.add_to_cart', product_id=product.id) }}" method="POST" class="d-flex align-items-center">
  <input class="form-control text-center me-3" name="quantity" type="number" value="1" min="1" style="max-width: 4rem;" />
  <button class="btn btn-primary flex-shrink-0" type="submit">
    <i class="bi-cart-fill me-1"></i> Add to cart
  </button>
</form>
```

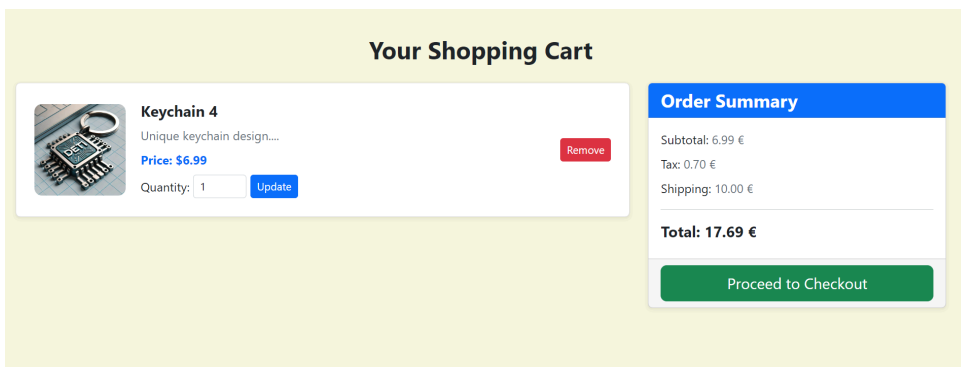
All forms do not have a CSRF token, so the system cannot validate the authenticity of all posting requests.

Exploitation of the Vulnerability:

Adding products to a cart:

An attacker can exploit this vulnerability by embedding a form or link on a malicious website that sends a request to the vulnerable application. For example:

"<form action="http://localhost:5000/cart/add/12" method="POST"> <input type="submit" value="Remove Item"> </form>"



Vulnerability Fix:

To fix this vulnerability, a **CSRF token was included** in **all** forms that perform state-changing actions such as updating cart items, removing items. Flask-WTF was used to generate and validate these tokens automatically.

```
<form action="{{ url_for('cart.add_to_cart', product_id=product.id) }}" method="POST" class="d-flex align-items-center">
  {{ csrf_token() }}
  <input class="form-control text-center me-3" name="quantity" type="number" value="1" min="1" style="max-width: 4rem;" />
  <button class="btn btn-primary flex-shrink-0" type="submit">
    <i class="bi-cart-fill me-1"></i> Add to cart
  </button>
</form>
```

This ensures that only requests with valid CSRF tokens, generated by the application and sent with the user's session, will be processed.